

# PHISH DETECTOR WORKFLOW AND USAGE GUIDE

Version 1.0.0 - Generated by scripts/generate\_workflow\_pdf.py

## PROJECT OVERVIEW

Phish Detector is a phishing URL analysis toolkit with three first-class entry points: a FastAPI backend with Celery workers, a Typer based CLI for power users, and a Manifest v3 browser extension that surfaces risk verdicts directly inside the browser. All components share a PostgreSQL datastore so analysis history is consistent regardless of the client.

The repository provides a docker-compose stack for local development and bundles infrastructure defaults (Redis, Selenium, PostgreSQL) alongside the Python services and web assets. This document captures the recommended workflow for contributors and end users.

## SYSTEM ARCHITECTURE

The system starts with 'backend/', a FastAPI application that exposes '/api/analyze', '/api/result/{task\_id}', and '/api/history'. Incoming requests enqueue Celery tasks which perform heuristic scoring, domain age lookups, and Selenium powered DOM inspection to flag risky form elements.

'cli/' hosts a Typer command line client that talks to the API, offering 'analyze' and 'history' commands with verdict aware output. 'extension/' contains a Chromium compatible popup that submits URLs, polls task status, and renders the persisted history table.

PostgreSQL stores every analysis record and Redis serves as the Celery broker and result backend. Selenium Grid (headless Chrome) enables lightweight page rendering and credential field detection.

## DEVELOPMENT WORKFLOW

Recommended steps when contributing to the repo:

- Install Docker and Docker Compose, then run 'docker-compose up --build' at least once to seed the database schema.
- Export environment overrides as needed ('DATABASE\_URL', 'REDIS\_URL', 'SELENIUM\_URL', 'CORS\_ALLOW\_ORIGINS').
- For backend changes, create a virtual environment and install 'backend/requirements.txt', then run 'uvicorn app.main:app --reload'.
- Run Celery locally with 'celery -A app.tasks worker --loglevel=info' while developing tasks.

- Execute targeted tests or linting ('pytest', 'ruff', etc.) if the project later adds them.  
Today, functional verification hinges on manual endpoint and client checks.

## RUNNING THE STACK

For the simplest local setup, rely on Docker Compose. The shipped file starts five services: postgres, redis, selenium, backend, and worker. The backend is accessible on `http://localhost:8000` after the containers settle.

When running natively, ensure PostgreSQL and Redis are installed and that Selenium (Standalone Chrome or Edge) is reachable at the configured 'SELENIUM\_URL'. Start the FastAPI app and Celery worker separately.

## CLI USAGE

The CLI defaults to 'http://localhost:8000'. Override with 'PHISH\_API\_URL' when targeting remote deployments.

- 'python cli/cli.py analyze https://example.com --verbose' submits a URL and streams the final verdict plus optional detail JSON.
- 'python cli/cli.py history --limit 20 --verdict phishing' lists the most recent results filtered by verdict.
- Network and polling exceptions surface inline; rerun the command once the backend is reachable.

## BROWSER EXTENSION USAGE

Load the extension via 'chrome://extensions' (Developer Mode -> Load unpacked -> select 'extension/'). Update the API base inside the popup if the backend runs on a different host.

The popup permits fetching the active tab URL, manual entry, and quick access to recent history. Results reuse the same verdict labels (SAFE, WARNING, PHISHING, UNKNOWN, FAILED) as the CLI.

Adjust 'CORS\_ALLOW\_ORIGINS' in the backend configuration and 'host\_permissions' in 'manifest.json' before pointing the extension at production domains.

## API REFERENCE

Key HTTP endpoints exposed by the backend:

- 'POST /api/analyze' - body: '{ "url": "https://target" }'. Returns 'task\_id' and initial

status (QUEUED).

- 'GET /api/result/{task\_id}' - returns task metadata, risk score, verdict, and heuristic details.
- 'GET /api/history?limit=20&offset=0&verdict=SAFE' - paginated list of persistent analysis records.

## DATA MODEL

The 'analysis\_results' table tracks task\_id, url, status, risk\_score, detail JSON, and timestamps. Celery creates or updates rows as tasks progress, marking failures to aid debugging.

JSON details capture heuristics (keyword matches, subdomain score), domain age metrics, Selenium outcomes, and screenshot path (when enabled).

## TESTING AND VERIFICATION

Before shipping changes, confirm the following:

- Backend endpoints respond with HTTP 200 using 'curl' or 'httpie'.
- Celery worker processes jobs without crashing and updates history records.
- CLI commands finish successfully against a running backend.
- Browser extension can submit URLs and refresh history without CORS errors.

## DEPLOYMENT CHECKLIST

For production rollouts:

- Provision PostgreSQL, Redis, and a Selenium compatible browser sandbox.
- Set secure environment variables and rotate credentials regularly.
- Front the FastAPI app with HTTPS (e.g., via Traefik or Nginx).
- Scale Celery workers based on expected concurrency, adding autoscaling if running in Kubernetes or ECS.
- Bake the CLI into automation pipelines (CI, Slack bots) where phishing URLs need on-demand evaluation.

## TROUBLESHOOTING

If tasks remain stuck in QUEUED or RUNNING, inspect the Celery worker logs and verify Redis connectivity. Selenium related errors often stem from mismatched driver versions or browsers failing to start in containerized environments.

CORS issues show up as blocked requests in the browser console. Update 'CORS\_ALLOW\_ORIGINS' and redeploy the backend to grant the requesting origin access.